Project 2
John Bryce
Rotem Amitay
s3
TMagen773636

The script begins by checking whether the current user has **root privileges**.

- If the user is **not root**, the script will display a message and exit.
- If the user **is root**, the script continues execution and calls the GETFILE function.

```bash
#!/bin/bash

# AutoDF - main script (colors + inline comments added)
START_TIME=''
END_TIME=''
HOME=''                    # will be set to working dir inside RUNSTRINGS
MEM_DUMP=''                # full path provided by user
OUT_DIR_NAME=''            # user-provided project name
OUT_DIR_PATH=''            # user-provided parent path for project
MEM_FILE=''                # basename of MEM_DUMP
NETWORK_FILE=''            # path to first found pcap/pcapng (if any)

# Color helper comment:
# tput setaf N -> set color; tput sgr0 -> reset color
# Color mapping used in this script: 1=red,2=green,3=yellow,4=blue,6=cyan

# Check the current user; exit if not root
function CHECKROOT()
{
    USER=$(whoami)
    if [ "$USER" != "root" ]; then
        tput setaf 1 # Red
        echo 'User is not root....Exiting....' # Important output: not root
        tput sgr0
        figlet "YOU SHALL NOT PASS!" # ASCII art for not root
        exit
    else
        tput setaf 2 # Green
        echo 'User is root!...Continuing...' # Important output: root
        tput sgr0
        figlet "YOU ARE ROOT!" # ASCII art for root
        GETFILE
    fi
}
```

```
┌──(CatNotFound㉿HOMEDESKTOP)-[/mnt/c/Users/lolam/OneDrive/Desktop/JohnBryce/PROJECTS/AutoDF]
└─$ sudo ./script.sh
[sudo] password for CatNotFound:
User is root!...Continuing...
```

The GETFILE function prompts the user to provide the **full path** to the memory file being investigated.
It then verifies that the specified file **actually exists**.

- If the file does not exist, the script notifies the user and exits.
- If the file is found, the script saves the full path into a variable.

```
# Allow the user to specify the filename; check if the file exists
function GETFILE()
{
    tput setaf 4 # Blue
    echo "Please provide *FULL PATH* of memory dump file." # Prompt for memory dump
    tput sgr0
    read MEM_DUMP

    # Validate memory dump file exists
    if [ ! -f "$MEM_DUMP" ]; then
        tput setaf 1 # Red
        echo "File does not exist: $MEM_DUMP" # Error: file not found
        tput sgr0
        exit 1
    fi
```

```
Please provide *FULL PATH* of memory dump file.
/mnt/c/Users/lolam/OneDrive/Desktop/JohnBryce/PROJECTS/AutoDF/memoryfile.mem
File does not exist: /mnt/c/Users/lolam/OneDrive/Desktop/JohnBryce/PROJECTS/AutoDF/memoryfile.mem

Please provide *FULL PATH* of memory dump file.
/mnt/c/Users/lolam/OneDrive/Desktop/JohnBryce/PROJECTS/AutoDF/memdump.mem
Please provide desired name for output directory for the script
```

Next, the user is asked to:

- Choose a **name** for the output directory.
- Specify the **location** where the directory will be created.

```
Please provide desired name for output directory for the script
ProjectDump
Please provide desired *FULL PATH* location for output directory for the script.
/mnt/c/Users/lolam/OneDrive/Desktop/JohnBryce/PROJECTS/AutoDF
Directory Created...
/mnt/c/Users/lolam/OneDrive/Desktop/JohnBryce/PROJECTS/AutoDF/ProjectDump
```

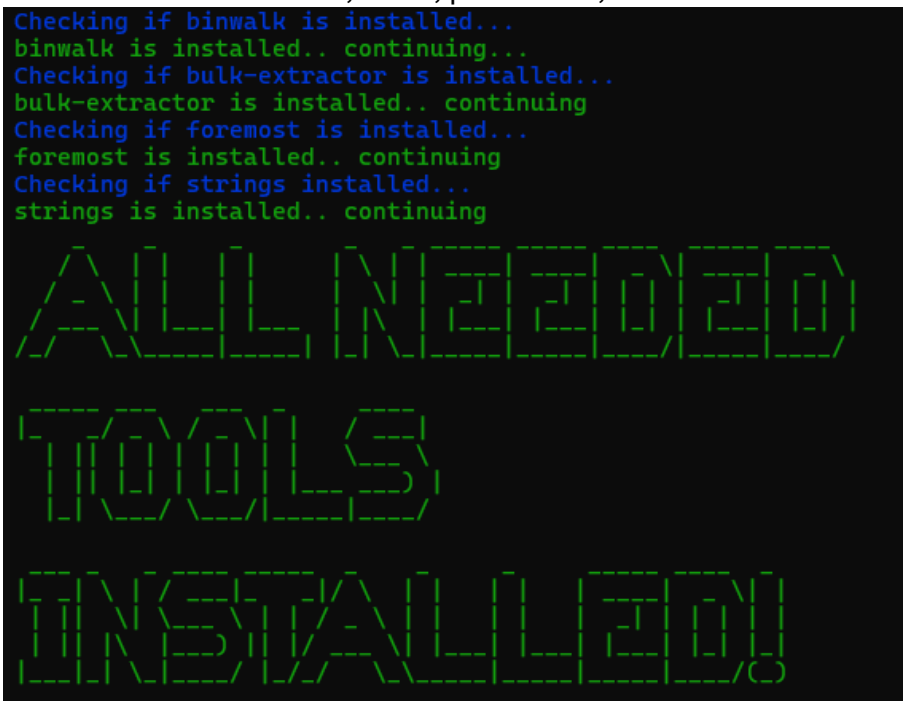
```
    tput setaf 4 # Blue
    echo "Please provide desired name for output directory for the script" # Prompt for output dir name
    tput sgr0
    read OUT_DIR_NAME

    tput setaf 4 # Blue
    echo "Please provide desired *FULL PATH* location for output directory for the script." # Prompt for output dir path
    tput sgr0
    read OUT_DIR_PATH

    # Change to parent / create project directory if needed
    cd "$OUT_DIR_PATH" || { tput setaf 1; echo "Cannot cd to $OUT_DIR_PATH"; tput sgr0; exit 1; }

    if [ -d "$OUT_DIR_NAME" ]; then
        tput setaf 3 # Yellow
        echo "Directory $OUT_DIR_NAME already exists in $OUT_DIR_PATH." # Warn: dir exists
        tput sgr0
    else
        tput setaf 2 # Green
        mkdir -p "$OUT_DIR_NAME"      # create directory safely
        echo "Directory Created..." # Success: dir created
        tput sgr0
    fi
```

After the output directory is created, the script moves the memory file into it and proceeds to the **tools installation phase**.

```
    # move memdump into project dir and cd there
    mv "$MEM_DUMP" "$OUT_DIR_PATH/$OUT_DIR_NAME"
    cd "$OUT_DIR_PATH/$OUT_DIR_NAME" || exit 1
    tput setaf 6 # Cyan
    pwd # Show current path (detail)
    tput sgr0
    GETTOOLS
}
```

The GETTOOLS function checks whether each required forensic tool is installed by verifying that its execution command exists.
If a tool is missing, the script automatically installs it.
This check is performed for the following tools:

- **Binwalk** – Used to analyze binary and firmware files by identifying embedded files, compressed data, and hidden content.
- **Bulk Extractor** – Extracts digital artifacts such as emails, URLs, IP addresses, and other patterns directly from disk images or memory files without relying on the file system.
- **Foremost** – Recovers files (images, documents, archives, etc.) from disk images based on file headers and footers, even if the file system is damaged or deleted.
- **Strings** – Extracts readable ASCII and Unicode strings from binary files, which may reveal commands, URLs, passwords, or other useful information.



```
# Install forensics tools if missing
function GETTOOLS()
{
    tput setaf 4 # Blue
    echo "Checking if binwalk is installed..." # Checking binwalk
    tput sgr0
    sleep 1
    if ! command -v binwalk > /dev/null; then
        tput setaf 1 # Red
        echo "Binwalk not found...Installing...." # Installing binwalk
        tput sgr0
        sudo apt install binwalk -y
        sleep 2
    else
        tput setaf 2 # Green
        echo "binwalk is installed.. continuing..." # Binwalk installed
        tput sgr0
        sleep 2
    fi
```

Volatility Installation

The script also uses **Volatility**, a tool designed to extract information from memory dumps, such as:

- Running processes
- Network connections
- Open files
- Other memory-resident artifacts

Volatility has a different installation process.

Since the script could not reliably download Volatility from an external server, the installation files are included locally.

The script simply moves the Volatility executable into the directory where the script is running, which is the **output directory** created earlier.

```
function VOLSETUP()
{
    tput setaf 4 # Blue
    echo "Seting up Volatility... " # Info: copy volatility helper
    tput sgr0

    # Copy the standalone volatility binary into the working dir
    cd "$OUT_DIR_PATH/Volatililty_for_Linux/volatility_2.5.linux.standalone" || exit 1
    sleep 1
    cp "$OUT_DIR_PATH/Volatililty_for_Linux/volatility_2.5.linux.standalone/vol" "$OUT_DIR_PATH/$OUT_DIR_NAME"
    cd "$OUT_DIR_PATH/$OUT_DIR_NAME" || exit 1
    figlet "VOLATILITY IS ALL SET!"
    RUNSTRINGS
}
```

After Volatility is set up, the script will run the first tool function, Which is RUNSTRINGS.

RUNSTRINGS Function

Once Volatility is set up, the script starts its analysis by calling the RUNSTRINGS function. This function:

1. Saves the memory file name into a variable.
2. Performs a full strings scan on the memory file.
3. Saves all human-readable output into a file.

```
function RUNSTRINGS()
{
    HOME=$(pwd) # set HOME for strings outputs to working dir
    MEM_FILE=$(basename "$MEM_DUMP") # get only filename
    tput setaf 6 # Cyan
    echo "$MEM_FILE" # Show memory dump filename (detail)
    tput sgr0

    tput setaf 4 # Blue
    echo "Creating strings output directory" # Creating output dir
    tput sgr0
    sleep 1
    mkdir -p STRINGS_DUMP # use -p to avoid errors

    tput setaf 4 # Blue
    echo "Running full strings scan on $MEM_FILE" # Running strings scan
    tput sgr0
    strings "$MEM_FILE" > "$HOME/STRINGS_DUMP/strings-full.txt"
    tput setaf 2 # Green
    echo "[*SCAN COMPLETE*]" # Scan complete
    tput sgr0
    ls "$HOME/STRINGS_DUMP"
```

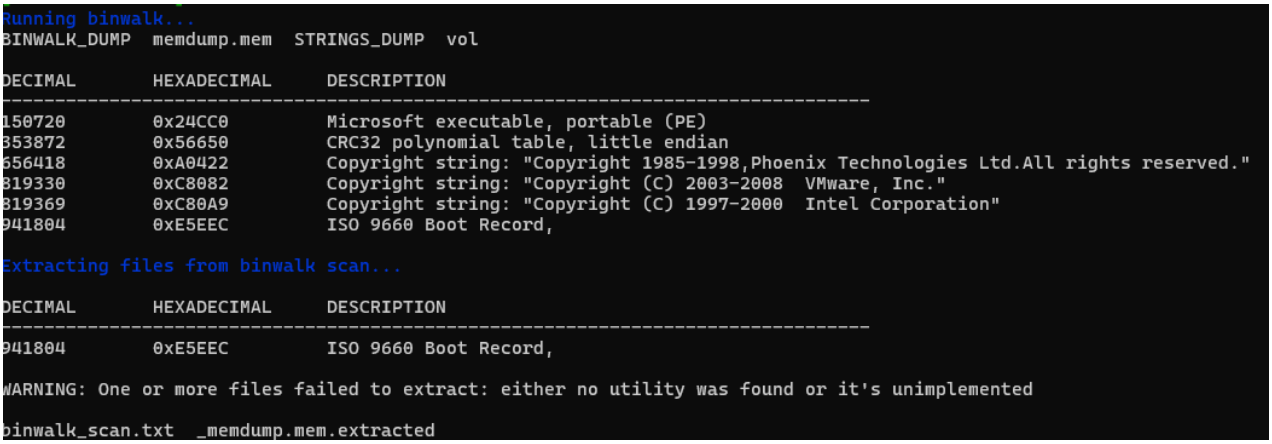
After that it will start scanning based on these keywoards:
-username,
-password,
-address,
-user,
-IP,
-connect,
-network,
-.exe

```
# Search and save specific patterns (username, password, address, etc.)
tput setaf 4; echo "Scanning for username in $MEM_FILE..."; tput sgr0
sleep 1
STRINGS_USERNAME=$(strings "$MEM_FILE" | grep -i 'username' || true)
echo "$STRINGS_USERNAME" > "$HOME/STRINGS_DUMP/strings-username.txt"
ls "$HOME/STRINGS_DUMP"
tput setaf 2; echo "[*SCAN COMPLETE*]"; tput sgr0
sleep 1
```

```
memdump.mem
Creating strings output directory
Running full strings scan on memdump.mem
[*SCAN COMPLETE*]
strings-full.txt
Scanning for username in memdump.mem...
strings-full.txt  strings-username.txt
[*SCAN COMPLETE*]
Scanning for password in memdump.mem...
strings-full.txt  strings-password.txt  strings-username.txt
[*SCAN COMPLETE*]
Scanning for address in memdump.mem...
strings-address.txt  strings-full.txt  strings-password.txt  strings-username.txt
[*SCAN COMPLETE*]
Scanning for user in memdump.mem...
strings-address.txt  strings-full.txt  strings-password.txt  strings-username.txt  strings-user.txt
[*SCAN COMPLETE*]
Scanning for IP in memdump.mem...
strings-address.txt  strings-full.txt  strings-IP.txt  strings-password.txt  strings-username.txt  strings-user.txt
[*SCAN COMPLETE*]
Scanning for connect in memdump.mem...
strings-address.txt  strings-full.txt  strings-password.txt  strings-user.txt
strings-connect.txt  strings-IP.txt    strings-username.txt
[*SCAN COMPLETE*]
Scanning for network in memdump.mem...
strings-address.txt  strings-full.txt  strings-network.txt   strings-username.txt
strings-connect.txt  strings-IP.txt    strings-password.txt  strings-user.txt
[*SCAN COMPLETE*]
Scanning for .exe in memdump.mem...
strings-address.txt  strings-exe.txt   strings-IP.txt       strings-password.txt  strings-user.txt
strings-connect.txt  strings-full.txt  strings-network.txt  strings-username.txt
[*SCAN COMPLETE*]
```

The script will then save each scan in its own file and save them to the STRINGS_DUMP directory it created.

Once RUNSTRINGS has finished, RUNBINWALK will be called.
The RUNBINWALK will scan the memory file and attempt to extract any hidden or imbedded files it may find.




```
function RUNBINWALK()
{
    HOME=$(pwd)                    # ensure HOME points to working dir
    MEM_FILE=$(basename "$MEM_DUMP")
    tput setaf 4; echo "Running binwalk..." ; tput sgr0
    sleep 1
    mkdir -p BINWALK_DUMP
    ls
    sleep 2
    binwalk "$MEM_FILE" > "$HOME/BINWALK_DUMP/binwalk_scan.txt"
    cat "$HOME/BINWALK_DUMP/binwalk_scan.txt"
    tput setaf 4; echo "Extracting files from binwalk scan..." ; tput sgr0
    sleep 2
    binwalk -e -C "$HOME/BINWALK_DUMP" --run-as=root "$MEM_FILE"
    ls "$HOME/BINWALK_DUMP"
    sleep 2
    RUNBULK
}
```

The RUNBULK function will extract all artifacts from the memory file and will also look for network files (.pcap) and will notify the user about location and size of network file if found.

```
Running bulk-extractor
mkdir "BULK_DUMP"
bulk_extractor version: 2.1.1
Input file: "memdump.mem"
Output directory: "BULK_DUMP"
Disk Size: 536870912
Scanners: aes base64 elf evtx exif facebook find gzip httplogs json kml_carved msxml net ntfsindx ntfslogfile ntfsmft ntfsusn pdf rar sqlite utmp vca
rd_carved windirs winlnk winpe winprefetch zip accts email gps
Threads: 16
going multi-threaded...( 16 )
bulk_extractor      Sat Dec 13 14:13:18 2025

available_memory: 7674580992
bytes_queued: 0
depth0_bytes_queued: 0
depth0_sbufs_queued: 0
elapsed_time:  0:00:00
estimated_date_completion: 2025-12-13 14:13:17
estimated_time_remaining: n/a
fraction_read: 0.000000 %
max_offset: 0
sbufs_created: 0
sbufs_queued: 0
sbufs_remaining: 0
Time producer spent waiting for scanners to process data:         0:00:01 (1.55 seconds)
Time consumer scanners spent waiting for data from producer:      0:01:31 (91.05 seconds)
Average time each consumer spent waiting for data from producer:  0:00:00 (0.00 seconds)
*** More time spent waiting for workers. You need a faster CPU or more cores for improved performance.
Total email features found: 123
bulk-extractor completed..PATH: /mnt/c/Users/lolam/OneDrive/Desktop/JohnBryce/PROJECTS/AutoDF/ProjectDump/BULK_DUMP
Looking for network file...
Network File *FOUND* Location: /mnt/c/Users/lolam/OneDrive/Desktop/JohnBryce/PROJECTS/AutoDF/ProjectDump/BULK_DUMP/packets.pcap
File size: 103629
```

```
function RUNBULK()
{
    MEM_FILE=$(basename "$MEM_DUMP")                    # get filename from full path
    tput setaf 4                                        # blue
    echo "Running bulk-extractor"                       # info: starting bulk_extractor
    tput sgr0                                           # reset color
    sleep 1

    # bulk_extractor writes into a 'BULK_DUMP' directory relative to current working dir
    bulk_extractor -o BULK_DUMP "$MEM_FILE"

    tput setaf 2                                        # green
    echo "bulk-extractor completed..PATH: $OUT_DIR_PATH/$OUT_DIR_NAME/BULK_DUMP"  # success + path
    tput sgr0                                           # reset color
    sleep 1

    tput setaf 4                                        # blue
    echo "Looking for network file..."                  # info: searching for pcap/pcapng
    tput sgr0                                           # reset color
    sleep 1

    # Search for the first .pcap or .pcapng file under project BULK_DUMP (absolute path)
    NETWORK_FILE=$(find "$OUT_DIR_PATH/$OUT_DIR_NAME/BULK_DUMP" -type f \( -iname '*.pcap' -o -iname '*.pcapng' \) -print -quit 2>/dev/null)

    if [ -n "$NETWORK_FILE" ]; then
        tput setaf 2                                    # green
        echo "Network File *FOUND* Location: $NETWORK_FILE"   # found message (full path)
        tput sgr0
        FILE_LISTING=$(ls -l -- "$NETWORK_FILE" | awk '{print $5}')   # capture file size (bytes) safely
        tput setaf 6                                    # cyan
        echo "File size: $FILE_LISTING"
        tput sgr0
    else
        tput setaf 1                                    # red
        echo "Network file not found"                   # not found message
        tput sgr0                                       # reset color
    fi

    RUNFOREMOST                                         # continue pipeline
}
```

The RUNFOREMOST function will be called next.
It will scan the memory file for any files inside it and will attempt to extract them and display them for the user.

```
Starting foremost...
Processing: memdump.mem
|******|
[*COMPLETE!*]
audit.txt  avi  bmp  dll  exe  gif  htm  wav
```

```
function RUNFOREMOST()
{
    tput setaf 4; echo "Starting foremost..." ; tput sgr0
    sleep 1
    foremost -i "$MEM_FILE" -o FOREMOST_DUMP
    tput setaf 2; echo "[*COMPLETE!*]" ; tput sgr0
    ls "$OUT_DIR_PATH/$OUT_DIR_NAME/FOREMOST_DUMP"
    sleep 3

    RUNVOL
}
```

The first thing the RUNVOL function does is to scan the memory file and establish a system OS profile.
The function will save any scan results to a directory it created called VOL_DUMP.

```
Running volatility...
Volatility Foundation Volatility Framework 2.5
INFO    : volatility.debug    : Determining profile based on KDBG search...
          Suggested Profile(s) : WinXPSP2x86, WinXPSP3x86 (Instantiated with WinXPSP2x86)
                     AS Layer1 : IA32PagedMemoryPae (Kernel AS)
                     AS Layer2 : FileAddressSpace (/mnt/c/Users/lolam/OneDrive/Desktop/JohnBryce/PROJECTS/AutoDF/ProjectDump/memdump.mem)
                      PAE type : PAE
                           DTB : 0x2fe000L
                          KDBG : 0x80545ae0L
          Number of Processors : 1
     Image Type (Service Pack) : 3
                KPCR for CPU 0 : 0xffdff000L
             KUSER_SHARED_DATA : 0xffdf0000L
           Image date and time : 2012-07-22 02:45:08 UTC+0000
     Image local date and time : 2012-07-21 22:45:08 -0400
Getting profile...
Volatility Foundation Volatility Framework 2.5
INFO    : volatility.debug    : Determining profile based on KDBG search...
WinXPSP2x86
```

```
function RUNVOL()
{
    tput setaf 4; echo "Running volatility..." ; tput sgr0
    sleep 2
    mkdir -p VOLATILITY_DUMP
    ./vol -f "$MEM_FILE" --output-file="$OUT_DIR_PATH/$OUT_DIR_NAME/VOLATILITY_DUMP/imageinfo.txt" imageinfo
    cat "$OUT_DIR_PATH/$OUT_DIR_NAME/VOLATILITY_DUMP/imageinfo.txt"

    tput setaf 4; echo "Getting profile..." ; tput sgr0
    sleep 1
    SYSPROF=$(./vol -f "$MEM_FILE" imageinfo | grep -i profile | awk '{print $4}' | tr -d ',') # detect profile
    echo "$SYSPROF"
    sleep 2
```

After RUNVOL has established a system profile, it will first scan for running processes.

```
Geting list of running proccesses....
Volatility Foundation Volatility Framework 2.5
Offset(V)  Name                    PID   PPID   Thds     Hnds   Sess  Wow64 Start                          Exit
---------- -------------------- ------ ------ ------ -------- ------ ------ ------------------------------ ------------------------------
0x823c89c8 System                    4      0     53      240 ------      0
0x822f1020 smss.exe                368      4      3       19 ------      0 2012-07-22 02:42:31 UTC+0000
0x822a0598 csrss.exe               584    368      9      326      0      0 2012-07-22 02:42:32 UTC+0000
0x82298700 winlogon.exe            608    368     23      519      0      0 2012-07-22 02:42:32 UTC+0000
0x81e2ab28 services.exe            652    608     16      243      0      0 2012-07-22 02:42:32 UTC+0000
0x81e2a3b8 lsass.exe               664    608     24      330      0      0 2012-07-22 02:42:32 UTC+0000
0x82311360 svchost.exe             824    652     20      194      0      0 2012-07-22 02:42:33 UTC+0000
0x81e29ab8 svchost.exe             908    652      9      226      0      0 2012-07-22 02:42:33 UTC+0000
0x823001d0 svchost.exe            1004    652     64     1118      0      0 2012-07-22 02:42:33 UTC+0000
0x821dfda0 svchost.exe            1056    652      5       60      0      0 2012-07-22 02:42:33 UTC+0000
0x82295650 svchost.exe            1220    652     15      197      0      0 2012-07-22 02:42:35 UTC+0000
0x821dea70 explorer.exe           1484   1464     17      415      0      0 2012-07-22 02:42:36 UTC+0000
0x81eb17b8 spoolsv.exe            1512    652     14      113      0      0 2012-07-22 02:42:36 UTC+0000
0x81e7bda0 reader_sl.exe          1640   1484      5       39      0      0 2012-07-22 02:42:36 UTC+0000
0x820e8da0 alg.exe                 788    652      7      104      0      0 2012-07-22 02:43:01 UTC+0000
0x821fcda0 wuauclt.exe            1136   1004      8      173      0      0 2012-07-22 02:43:46 UTC+0000
0x8205bda0 wuauclt.exe            1588   1004      5      132      0      0 2012-07-22 02:44:01 UTC+0000
```

```
tput setaf 4; echo "Geting list of running proccesses...." ; tput sgr0
sleep 1
./vol -f "$MEM_FILE" --profile="$SYSPROF" --output-file="$OUT_DIR_PATH/$OUT_DIR_NAME/VOLATILITY_DUMP/pslist.txt" pslist
cat "$OUT_DIR_PATH/$OUT_DIR_NAME/VOLATILITY_DUMP/pslist.txt"
sleep 1
```

After that RUNVOL will choose what network command to use based on the system profile established.

```
Getting list of network connections....
Invalid profile for netscan, trying connections instead....
Volatility Foundation Volatility Framework 2.5
Offset(V)  Local Address             Remote Address            Pid
---------- ------------------------ ------------------------ ---
0x81e87620 172.16.112.128:1038      41.168.5.140:8080        1484
```

```
tput setaf 4; echo "Getting list of network connections...." ; tput sgr0
if [ "$SYSPROF" != "WinXPSP2x86" ] && [ "$SYSPROF" != "Win2003SP2x64" ]; then
    ./vol -f "$MEM_FILE" --profile="$SYSPROF" --output-file="$OUT_DIR_PATH/$OUT_DIR_NAME/VOLATILITY_DUMP/netscan.txt" netscan
    cat "$OUT_DIR_PATH/$OUT_DIR_NAME/VOLATILITY_DUMP/netscan.txt"
else
    tput setaf 3; echo "Invalid profile for netscan, trying connections instead...." ; tput sgr0
    ./vol -f "$MEM_FILE" --profile="$SYSPROF" --output-file="$OUT_DIR_PATH/$OUT_DIR_NAME/VOLATILITY_DUMP/connections.txt" connections
    cat "$OUT_DIR_PATH/$OUT_DIR_NAME/VOLATILITY_DUMP/connections.txt"
fi
```

Lastly RUNVOL will scan for any registry files and attempt to extract them.

```
Looking for registery information....
Volatility Foundation Volatility Framework 2.5
Virtual    Physical   Name
---------- ---------- ----
0xe18e5b60 0x093f8b60 \Device\HarddiskVolume1\Documents and Settings\Robert\Local Settings\Application Data\Microsoft\Windows\UsrClass.dat
0xe1a19b60 0x0a5a9b60 \Device\HarddiskVolume1\Documents and Settings\Robert\NTUSER.DAT
0xe18398d0 0x08a838d0 \Device\HarddiskVolume1\Documents and Settings\LocalService\Local Settings\Application Data\Microsoft\Windows\UsrClass.dat
0xe18614d0 0x08e624d0 \Device\HarddiskVolume1\Documents and Settings\LocalService\NTUSER.DAT
0xe183bb60 0x08e2db60 \Device\HarddiskVolume1\Documents and Settings\NetworkService\Local Settings\Application Data\Microsoft\Windows\UsrClass.dat
0xe17f2b60 0x08519b60 \Device\HarddiskVolume1\Documents and Settings\NetworkService\NTUSER.DAT
0xe1570510 0x07669510 \Device\HarddiskVolume1\WINDOWS\system32\config\software
0xe1571008 0x0777f008 \Device\HarddiskVolume1\WINDOWS\system32\config\default
0xe15709b8 0x076699b8 \Device\HarddiskVolume1\WINDOWS\system32\config\SECURITY
0xe15719e8 0x0777f9e8 \Device\HarddiskVolume1\WINDOWS\system32\config\SAM
0xe13ba008 0x02e4b008 [no name]
0xe1035b60 0x02ac3b60 \Device\HarddiskVolume1\WINDOWS\system32\config\system
0xe102e008 0x02a7d008 [no name]
```

```
    tput setaf 4; echo "Looking for registery information...." ; tput sgr0
    ./vol -f "$MEM_FILE" --profile="$SYSPROF" --output-file="$OUT_DIR_PATH/$OUT_DIR_NAME/VOLATILITY_DUMP/hivelist.txt" hivelist
    cat "$OUT_DIR_PATH/$OUT_DIR_NAME/VOLATILITY_DUMP/hivelist.txt"
```

```
    tput setaf 4; echo "Extracting registery files...." ; tput sgr0
    mkdir -p "$OUT_DIR_PATH/$OUT_DIR_NAME/VOLATILITY_DUMP/REGDUMP"
    ./vol -f "$MEM_FILE" --profile="$SYSPROF" --dump-dir="$OUT_DIR_PATH/$OUT_DIR_NAME/VOLATILITY_DUMP/REGDUMP" dumpregistry
    ls "$OUT_DIR_PATH/$OUT_DIR_NAME/VOLATILITY_DUMP/REGDUMP"
    sleep 2

    GENERATE_REPORT
}
```

```
Extracting registery files....
Volatility Foundation Volatility Framework 2.5
**************************************************
Writing out registry: registry.0xe18e5b60.UsrClassdat.reg

**************************************************
**************************************************
Writing out registry: registry.0xe1a19b60.NTUSERDAT.reg

**************************************************
**************************************************
Writing out registry: registry.0xe102e008.no_name.reg

**************************************************
**************************************************
Writing out registry: registry.0xe1571008.default.reg

**************************************************
**************************************************
Writing out registry: registry.0xe18398d0.UsrClassdat.reg

**************************************************
**************************************************
Writing out registry: registry.0xe1570510.software.reg

**************************************************
**************************************************
Writing out registry: registry.0xe17f2b60.NTUSERDAT.reg

**************************************************
```

```
**************************************************
registry.0xe102e008.no_name.reg   registry.0xe15709b8.SECURITY.reg   registry.0xe18398d0.UsrClassdat.reg  registry.0xe1a19b60.NTUSERDAT.reg
registry.0xe1035b60.system.reg    registry.0xe1571008.default.reg    registry.0xe183bb60.UsrClassdat.reg
registry.0xe13ba008.no_name.reg   registry.0xe15719e8.SAM.reg        registry.0xe18614d0.NTUSERDAT.reg
registry.0xe1570510.software.reg  registry.0xe17f2b60.NTUSERDAT.reg  registry.0xe18e5b60.UsrClassdat.reg
```

The last steps of the script is Generates a **report** by reading all files produced during execution and summarizing their contents.

```
Report written to: /mnt/c/Users/lolam/OneDrive/Desktop/JohnBryce/PROJECTS/AutoDF/ProjectDump/REPORT.txt
======================================
AutoDF Findings Report
Generated: 2025-12-13 14:15:09
======================================

Start time: 2025-12-13 14:13:52.991806600 +0200
End time:   2025-12-13 14:15:09

Memory dump
  Name: memdump.mem
  Path: /mnt/c/Users/lolam/OneDrive/Desktop/JohnBryce/PROJECTS/AutoDF/ProjectDump/memdump.mem
  Size: 512M

Overall output directory
  Path: /mnt/c/Users/lolam/OneDrive/Desktop/JohnBryce/PROJECTS/AutoDF/ProjectDump
  Total size: 1.3G

Per-tool summaries:
  STRINGS_DUMP
    Size: 8.4M
    Files: 9
  BINWALK_DUMP
    Size: 512M
    Files: 2
  BULK_DUMP
    Size: 216M
    Files: 3183
  FOREMOST_DUMP
    Size: 23M
    Files: 181
  VOLATILITY_DUMP
    Size: 15M
```

```
function GENERATE_REPORT()
{
    REPORT_FILE="$OUT_DIR_PATH/$OUT_DIR_NAME/REPORT.txt"
    mkdir -p "$OUT_DIR_PATH/$OUT_DIR_NAME"

    START_TIME=${START_TIME:-$(stat -c %y "$OUT_DIR_PATH/$OUT_DIR_NAME" 2>/dev/null || date '+%Y-%m-%d %H:%M:%S')}
    END_TIME=$(date '+%Y-%m-%d %H:%M:%S')

    {
        echo "========================================"
        echo "AutoDF Findings Report"
        echo "Generated: $END_TIME"
        echo "========================================"
        echo
        echo "Start time: $START_TIME"
        echo "End time:   $END_TIME"
        echo

        echo "Memory dump"
        MEM_PATH="$OUT_DIR_PATH/$OUT_DIR_NAME/$MEM_FILE"
        echo "  Name: $MEM_FILE"
        if [ -f "$MEM_PATH" ]; then
            du -h --max-depth=0 "$MEM_PATH" 2>/dev/null | awk '{print "  Path: " $2 "\n  Size: " $1}'
        else
            echo "  Path: $MEM_PATH (not found)"
        fi
        echo

        echo "Overall output directory"
        du -sh "$OUT_DIR_PATH/$OUT_DIR_NAME" 2>/dev/null | awk '{print "  Path: " $2 "\n  Total size: " $1}'
        echo

        echo "Per-tool summaries:"
        for DIR in STRINGS_DUMP BINWALK_DUMP BULK_DUMP FOREMOST_DUMP VOLATILITY_DUMP VOLATILITY_DUMP/REGDUMP; do
            FULL="$OUT_DIR_PATH/$OUT_DIR_NAME/$DIR"
            if [ -d "$FULL" ]; then
                echo "  $DIR"
                du -sh "$FULL" 2>/dev/null | awk '{print "    Size: " $1}'
                echo -n "    Files: "
                find "$FULL" -type f 2>/dev/null | wc -l | awk '{print $1}'
            fi
        done
        echo

        echo "Top 10 largest files in output dir:"
        du -ah "$OUT_DIR_PATH/$OUT_DIR_NAME" 2>/dev/null | sort -hr | head -n 10
        echo

        echo "Network capture(s) (pcap/pcapng) found:"
        NETWORK_FILE=${NETWORK_FILE:-$(find "$OUT_DIR_PATH/$OUT_DIR_NAME/BULK_DUMP" -type f \( -iname '*.pcap' -o -iname '*.pcapng' \) -print -quit 2>/dev/null)}
        if [ -n "$NETWORK_FILE" ]; then
            du -h "$NETWORK_FILE" 2>/dev/null | awk '{print "  " $2 " : " $1}'
        else
            echo "  None found"
        fi
        echo

        echo "Volatility information (if available):"
        IMGINFO="$OUT_DIR_PATH/$OUT_DIR_NAME/VOLATILITY_DUMP/imageinfo.txt"
        if [ -f "$IMGINFO" ]; then
            echo "  imageinfo (excerpt):"
            sed -n '1,40p' "$IMGINFO" | sed 's/^/    /'
            echo
        else
            echo "  imageinfo not available"
            echo
        fi

        echo "Counts by common extensions (excerpt):"
        for EXT in txt log pcap pcapng exe dll jpg png gif sh py; do
            CNT=$(find "$OUT_DIR_PATH/$OUT_DIR_NAME" -type f -iname "*.$EXT" 2>/dev/null | wc -l)
            [ "$CNT" -gt 0 ] && echo "  .$EXT : $CNT"
        done
        echo

        echo "Quick file age summary (newest 5 files):"
        find "$OUT_DIR_PATH/$OUT_DIR_NAME" -type f -printf '%TY-%Tm-%Td %TH:%TM:%TS %p\n' 2>/dev/null | sort -r | head -n 5 | sed 's/^/  /'
        echo

        echo "End of report."
    } > "$REPORT_FILE"

    tput setaf 6 2>/dev/null || true
    echo "Report written to: $REPORT_FILE"
    tput sgr0 2>/dev/null || true

    # Print report to stdout for immediate inspection
    cat "$REPORT_FILE"

    ZIPOUTPUT
}
```

Compresses the entire output directory into a single archive for easy storage and transfer.

```
function ZIPOUTPUT()
{
    cd "$OUT_DIR_PATH" || exit 1
    zip -r "$OUT_DIR_NAME.zip" "$OUT_DIR_PATH"
    ls
    RESETLAB
}
```